ISE Project Coding Standards

Team Guidelines

November 22, 2025

Quick Reference Table

Practice	Guideline
Naming	`lowerCamelCase` for all variables, functions, files, and class names. Example: `handleUserInput`, `chatbot.cpp`
Function Comments	Every function has a 2–4 line Javadoc comment block describing *purpose*, parameters (`@param`), return value (`@return`), and any edge details.
Header Guards	Every header file (`.h`) must use `#ifndef`, `#define`, `#endif` format with a unique macro.
Formatting	Use 4 spaces for indentation. Place opening braces on the same line as the loop or function.
Getters/Setters	Simple getters and setters must be defined inline in header files for consistency.
File Size Limit	Keep files under 500 lines. If exceeding, split into logical modules or helper files.
Commit Messages	Use conventional commit format: `type: brief description`. Types: `feat`, `fix`, `refactor`, `docs`, `test`.
Code Review	Review all code before push; comment using `TODO:` and `FIXME:` for improvements.

Function Comment Example (Javadoc Style)

```
/**
 * @brief Finds and prints the chatbot's response for user input
 *
 * Searches the response table for matching input and displays
 * the corresponding bot message. Falls back to default response
 * if no match found.
```

```
 *
 * @param input The user's typed message
 * @return true if response found, false otherwise
 */
```

Commit Message Format (Conventional Commits)

Follow the industry-standard Conventional Commits format for consistency and clarity:

Format: `type: brief description in lowercase`

Type	When to Use	Example
`feat`	New features or functionality	`feat: add document upload validation`
`fix`	Bug fixes and corrections	`fix: resolve CNIC validation error`
`refactor`	Code restructuring without changing behavior	`refactor: extract state handlers to separate module`
`docs`	Documentation changes only	`docs: update API documentation`
`test`	Adding or modifying tests	`test: add validation test cases`
`style`	Code formatting changes only	`style: apply consistent indentation`
`chore`	Build/config changes	`chore: update compilation command`

Pull Request Titles: Use the same format but with proper capitalization:

`Refactor: Extract state handlers to separate module`

`Fix: Resolve CNIC validation error`

`Feat: Add document upload validation`

Organizing Code: Header vs. Source Files

To keep our C++ codebase readable, maintainable, and easy for everyone to contribute, follow these practices:

- **Header files (.h):**
  - Write only **class/struct definitions** (data members and function signatures).
  - These files describe the public interface: what the class is and what it offers.
  - Add Javadoc comments above each class and public function, explaining purpose, key parameters (`@param`), and return value (`@return`).

  - **Simple getters and setters** should be defined inline in the header for consistency and performance.
- **Source files (.cpp):**
  - Provide the actual code for complex functions declared in the header.
  - Use inline or block comments inside functions when logic is tricky.
  - Always start by including the matching header (e.g., `#include "fileio.h"`).
  - **Keep files under 500 lines.** If exceeding, split into logical modules.
- **Why this split?** Keeps projects organized, speeds up teamwork, and prevents accidental code duplication. Changing a function's logic won't require recompiling all code that uses the interface.

Getter/Setter Example in Header:

```
class Application {
private:
    string fullName;
    int age;

public:
    // Getters - defined inline in header
    string getFullName() const { return fullName; }
    int getAge() const { return age; }

    // Setters - defined inline in header
    void setFullName(string n) { fullName = n; }
    void setAge(int a) { age = a; }

    // Complex functions - only declared, defined in .cpp
    void processApplication();
    bool validateData();
};
```

`std` **Namespaces and Best Practice:**

- **Never put** `using namespace std;` in a header file.
  - If you don't like repeating `std::`, you can write `using std::string;` or similar only at the top of your .cpp files.
  - In headers, explicitly use `std::` prefix for types like `std::string`, `std::vector`, `std::unordered_map`, etc.
- This avoids hidden bugs and naming conflicts in your project.

Summary Table:

File Type	What to put inside	Where to comment
.h Header	Class declarations, function signatures, inline getters/setters	Above each class/method with Javadoc
.cpp Source	Complex function implementations (not getters/setters)	Inside functions on tricky logic

Always keep interfaces (`.h`) and implementations (`.cpp`) separate. Javadoc comments in headers make it easy for teammates to use your code.

Code Modularity and Readability Guidelines

File Size Management:

- **Maximum 500 lines per file.** If a file grows beyond this, consider splitting it:
  - Extract helper functions to a separate utilities file
  - Separate related functionality into distinct modules
  - Create a manager class to coordinate complex operations
- Example: `chatbot.cpp` (1200 lines) split into:
  - `chatbot.cpp` (80 lines) - main loop
  - `state_handlers.cpp` (500 lines) - state logic
  - `application_input.cpp` (320 lines) - form collection

Function Size:

- Keep functions under 50 lines when possible
- If a function does multiple things, split it into helper functions
- One function should have one clear purpose

DRY Principle (Don't Repeat Yourself):

- If you copy-paste code more than once, create a helper function
- Use loops or helper functions instead of duplicating logic
- Extract common patterns into reusable utilities

When to Create a New Module:

- File exceeds 500 lines
- Code serves a distinct purpose (input, display, validation, etc.)
- Logic is reused across multiple files
- Code can be tested independently

Extra Tips

- Comment unfinished/problematic code with `TODO:` or `FIXME:`
- Keep functions focused: one main purpose, keep short when possible
- Group helpers and logic by feature, not one file per tiny function
- Use meaningful variable names - avoid single letters except for loops
- Add blank lines between logical sections for readability
- Always review your code and commit with clear, descriptive messages
- Before pushing, ask: "Can someone else understand this code easily?"

Standardization makes teamwork simple — stick to these for an A+ workflow!